CS3106L | Operating Systems Laboratory

Lab 8: Disk Scheduling and Rotational Latency

Laboratory practice • Hard-disk mechanics, rotational delay and disk scheduling

Objective

Explore why the request requiring the smallest head movement need not finish earliest. Use a deterministic HDD timing model to compare FCFS, shortest-seek-time-first (SSTF), LOOK, and shortest-access-time-first (SATF). The work is exploratory: there is no assessment or submission.

Background

Servicing a conventional rotating-disk request requires moving the head to the requested cylinder, waiting until the requested sector rotates beneath the head, and transferring its data. Rotation continues while the head moves. Therefore, a request with a short seek can still incur nearly a full revolution of rotational waiting.

1. Disk model

Quantity	Assumption
Cylinders	200, numbered 0 through 199; one head and one logical track per cylinder
Sectors per track	12, numbered 0 through 11
Rotation	120 ticks per revolution = 10 ticks per sector
Head movement	3 ticks per cylinder crossed; no additional settle time
Track skew	Physical angle of logical sector 0 on cylinder c is offset by c sectors (modulo 12)
Requests	(ID, arrival tick, cylinder, starting sector, length in sectors); no cross-track transfers
Execution	Non-preemptive; head and platter move only according to this simplified model

Track skew is an assumed angular offset between neighboring logical tracks, not extra seek time. The numerical geometry is illustrative rather than a specification of the storage device used by LUIT.

2. Timing equations

Let h be the current cylinder, c the target cylinder, s the starting sector, n the number of sectors, and t the dispatch time. All calculations use integer ticks:

- Seek time = $3 \times |c - h|$
- Angular phase after seek = $(t + \text{seek time}) \bmod 120$
- Target sector phase = $10 \times ((s + c) \bmod 12)$
- Rotational wait = $(\text{target sector phase} - \text{angular phase after seek}) \bmod 120$
- Transfer time = $10 \times n$
- Completion time = t + seek time + rotational wait + transfer time

The modulus in rotational wait returns a value from 0 to 119 ticks. Every request is selected from those that have already arrived. If none is ready, time advances to the next arrival without resetting the platter angle.

3. Worked example

In classic.json, the head starts at cylinder 50 at tick 0. Under FCFS, A (cylinder 82, sector 0, length 1) is followed by B (cylinder 170, sector 1, length 1).

Quantity	Request A	Request B
Dispatch tick	0	110
Seek	3 × 32 = 96	3 × 88 = 264
Phase after seek	96	(110 + 264) mod 120 = 14
Target phase	100	30
Rotational wait	4	16
Transfer	10	10
Completion tick	110	400

Check both completion times using the supplied FCFS simulator. Notice that the second request begins at tick 110, not at tick 0.

4. Laboratory exercises

A. Establish the baseline Run the baseline tests and FCFS simulation. Trace the first two completions in classic.json by hand and compare your calculations with the output.

B. Implement LOOK Complete select_look() in disklab/model.py. Select the nearest ready request in the current scan direction; reverse only when no ready request remains on that side. Do not move to the physical end cylinder merely to reverse.

C. Implement SATF Complete select_satf() using seek time + predicted rotational wait + transfer time at the current dispatch tick. Use earlier arrival time and then lexicographic request ID as tie-breakers.

D. Compare the policies Run all four schedulers on classic.json and rotation_trap.json. Compare service order, total seek ticks, mean response time, maximum response time, and final completion tick.

E. Construct a counterexample Create an original scenario of at least four requests where SSTF uses fewer total seek ticks but SATF achieves a lower mean response time. Explain the changing angular phase after each completed request.

F. Respect arrival times Use arrival_trap.json to explain why a request that arrives later cannot be chosen at an earlier dispatch. A newly arriving request cannot interrupt a request already in service.

G. Interpret the findings Discuss why SATF need not always outperform SSTF, how LOOK differs from SCAN, whether starvation is possible, and why HDD seek/rotation formulas are not applicable to SSDs.

5. Running the laboratory

Use Python 3 and the standard library. Run these commands from the student/ directory:

```
PYTHONPATH=. python3 -m unittest discover -s tests -p 'test_baseline.py' -v
PYTHONPATH=. python3 tools/run.py scenarios/classic.json --policy fcfs
```

After completing the two TODO functions:

```
PYTHONPATH=. python3 -m unittest discover -s tests -v
PYTHONPATH=. python3 tools/run.py scenarios/rotation_trap.json --policy sstf
PYTHONPATH=. python3 tools/run.py scenarios/rotation_trap.json --policy satf
```

The untouched starter passes the baseline checks but raises NotImplementedError for LOOK and SATF. This is expected; the complete test suite should pass after those two functions have been implemented correctly.

6. Discussion and interpretation

- Seek time and total access time are different metrics. Minimizing the former does not guarantee a minimum mean response time.
- SATF is greedy: it minimizes the estimated service time of the next selected request, not the globally optimal service order.
- The simulator labels the absolute final completion tick as makespan. This includes any idle time and is not the sum of busy durations.
- The model omits multiple heads, seek acceleration, zone-dependent sectors, controller caches and command reordering. No modifications to LUIT kernel files are needed.

7. Self-check questions

- At tick 0, the head is at cylinder 40. For a request at cylinder 48, sector 5, length 1, calculate the seek, rotational wait, transfer time and finish tick.
- Can a request with shorter seek time have longer total service time? Give a numerical example using the model above.
- Suppose the next request arrives at tick 75 but the disk is busy until tick 110. When can this request first be considered by the dispatcher?
- If two requests have equal SATF service estimates, which one wins and why? What happens if a policy attempts to choose an unarrived request?

Use OBSERVATION_SHEET.md if you wish to keep your calculations together during the lab. No written report or upload is required.